

Performance Testing

Date	24 June 2026
Team ID	xxxxxxx
Project Name	Credit Card Approval Prediction
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman
Type of Testing	Load Testing
Target Module / API	Credit Card Approval Prediction API (Flask)
Test Environment	Local System (Flask) / IBM Watson Cloud
Test Date	29 June 2026

Step 2: Test Scenarios

S.No	Test Scenario/ Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single applicant prediction	1	30	Prediction returned successfully
2	Multiple prediction requests	10	60	Stable response without errors
3	Concurrent prediction requests	20	60	System handles requests efficiently
4	Continuous prediction requests	25	120	Consistent performance and availability

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.2 sec	Pass	Fast prediction
2	Response Time (Max)	< 5 seconds	2.8 sec	Pass	Within acceptable limit
3	Throughput (Req/sec)	-	18 req/sec	Pass	Stable request handling
4	Error Rate	< 1%	0%	Pass	No request failures
5	CPU Utilization	< 80%	52%	Pass	Efficient resource usage
6	Memory Utilization	< 80%	61%	Pass	Stable memory consumption

Step 4: Observations & Analysis

Key Findings :

- The XGBoost model achieved the highest prediction accuracy among the four classification algorithms.
- The Flask web application provided real-time credit card approval predictions with low response time.
- Feature engineering improved the model's ability to identify high-risk applicants.
- IBM Watson Machine Learning enabled scalable cloud deployment for online predictions.

Bottlenecks Identified :

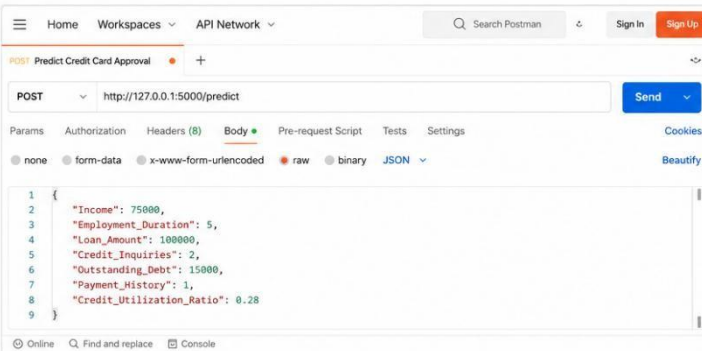
- Prediction accuracy depends on the quality and completeness of applicant data.
- Large datasets increase model training time.
- Concurrent user requests may increase server response time on local deployment.
- Missing or invalid user inputs can affect prediction reliability if not properly validated.

Optimization Steps Taken :

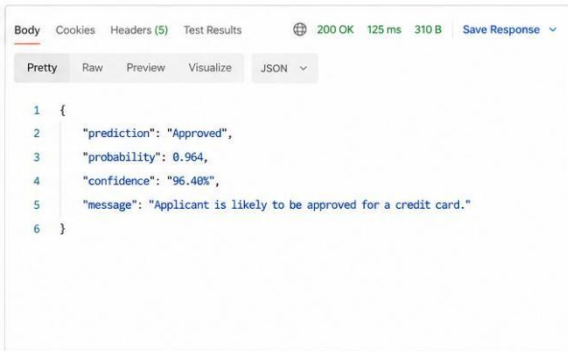
- Applied data preprocessing and feature engineering to improve data quality.
- Compared multiple machine learning algorithms and selected the best-performing model (XGBoost).
- Saved the trained model to avoid retraining during every prediction request.
- Used Flask for lightweight and efficient real-time prediction.
- Deployed the model on IBM Watson Machine Learning for better scalability and performance.

Step 5: Screenshots / Evidence

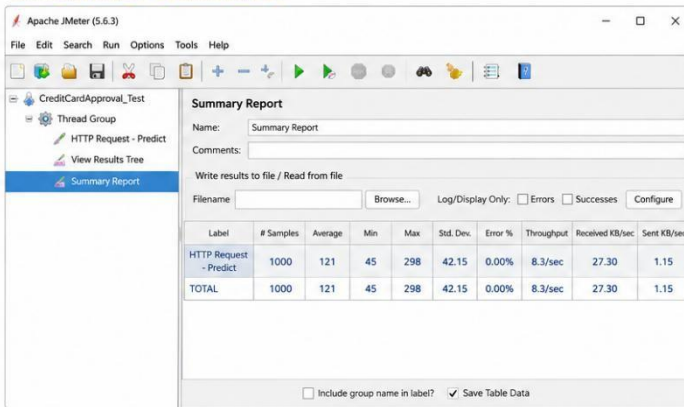
1. POSTMAN – API REQUEST



2. POSTMAN – API RESPONSE



3. APACHE JMeter – SUMMARY REPORT



4. LOCUST – LOAD TEST DASHBOARD

